

Applying QAOA to the Max-Cut Problem: A Study in Quantum Optimization

Anna Sehgal, Jasjit Dhillon, Alexander L. Liu, Matthew J. Hernandez

July 11, 2025

0 INTRODUCTION

The Quantum Approximate Optimization Algorithm (QAOA) is a **hybrid-classical algorithm** that aims to solve combinatorial optimization problems, which boil down to finding the optimal solution out of a large number of possibilities through the use of classical and quantum computation methods, through a hybrid approach.

Max-Cut is one of the classical NP-hard problems, where we are given a graph G with a certain number of nodes and have to divide the nodes into two groups using a “cut” that increases the number of edges between these groups. An example of this would be splitting a group into two different teams for a game, where each of the edges represents rivalry. We will ensure that we put the rivals on opposite teams to make the game more fun and get the highest number of rivals possible between the teams for a better and more exciting game. This problem is usually used to benchmark optimization problems; therefore, it is a perfect candidate for our experiment.

To contrast our algorithm with the classical approach, and test how well it works, we compare the outputs with the **Goemans-Williamson algorithm**, a well-known polynomial-time approximation algorithm with an $\sim 87.8\%$ chance of finding the best cut. In theory, QAOA, a hybrid approach, exceeds that number; however, it runs into a few blocks, particularly due to noisy hardware with limited circuit depth, against the established classical approach.

Our main focus is to implement QAOA for Max-Cut on small graphs and explore the effects on circuit depth p and the impact of simulated noise on the performance of this algorithm. We chose QAOA due to its specific design to work with today’s noisy hardware to some level and due to its shallow circuit depth and hybrid nature, making it highly adaptable and used for a wide range of combinatorial optimization problems, like Max Cut, Traveling Salesman Problem, etc.

1 METHODS AND SETUP

1.1 Tools and Platform Used

For the implementation of the Quantum Optimization Algorithm (QAOA), we used the Qiskit libraries such as AerSimulator. The implementation and testing were done in a Google Colab notebook, written in a Python-based framework for the creation of quantum circuits using Qiskit’s built-in simulators and visualization tools. The report was composed using Overleaf and Google Docs, and team communication was handled through Discord. The matplotlib library was used to produce the visualization, and the algorithm simulation was executed using the Aer-Simulator from qiskit-aer.

1.2 Problem Setup

We applied QAOA to solve the Max-Cut problem on a small, unweighted graph. Max-Cut problem includes the division of the nodes of a graph into two groups so that the number of edges (“cuts”) between the two groups is maximized. The graph was defined using the rustworkx and networkx libraries in Python and visualized to represent potential cuts. We worked with 3 different graph sizes of the graph, which are 5, 10, and 20 nodes. Our initial implementation was using a 5-node graph using edges: (0, 1), (0, 2), (0, 4), (1, 2), (2, 3), (3, 4).

The Max-Cut cost Hamiltonian was constructed using the Pauli-Z operator. Each edge, a ZZ term was added with a unit weight, which gives us a sparse Pauli operator that represents the input of the quantum circuit.

```
[ ] def build_max_cut_paulis(graph: rx.PyGraph) -> list[tuple[str, list[int], float]]:
    pauli_list = []
    for u, v in gr.Create a note ({}):
        weight = graph.get_edge_data(u, v)
        pauli_list.append(("ZZ", [u, v], weight))
    return pauli_list

max_cut_paulis = build_max_cut_paulis(graph)
cost_hamiltonian = SparsePauliOp.from_sparse_list(max_cut_paulis, num_qubits=n)
print("Cost Function Hamiltonian:\n", cost_hamiltonian)

Cost Function Hamiltonian:
SparsePauliOp(['IIIZZ', 'IIZIZ', 'ZIIIZ', 'IIZZI', 'IZZII', 'ZZIII'],
coeffs=[1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j, 1.+0.j])
```

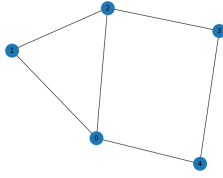


Fig. 1: 5-node graph used in the Max-Cut implementation. Each edge represents a connection where a cut can be placed.

1.3 QAOA Circuit Construction

The QAOA circuit was built using Qiskit's `qiskit.circuit.library` known as `QAOAAnsatz`, `wclass` with the cost Hamiltonian that is generated by the graph's adjacency matrix. This changes between a cost unitary and the applied standard traverse field mixing Hamiltonian, and uses the COBYLA optimizer for classical parameter optimization. We initially tested with depth $p = 2$ for the initial implementation using noiseless simulations. In addition, we are also testing with different depths, $p = 1, 2$, and 3 , as well as under ideal, noiseless conditions. We focused on $p = 1$ and $p = 3$ specifically for the noisy simulation to see the performance of this algorithm with depth, which is expected to degrade in performance due to depth causing decoherence. We simplified the scope to reduce circuit depth and mitigate noise effects on simulated NISQ hardware.

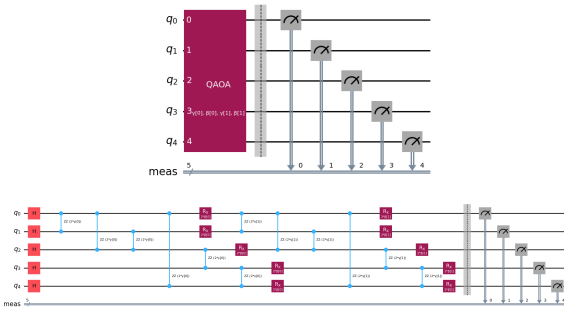


Fig. 2: QAOA circuit with depth $p = 2$, built using Qiskit for the 5-node Max-Cut problem. Hadamard gates initialize qubits, followed by cost and mixer layers.

1.4 Simulating Noisy and Noiseless Backends

Using Qiskit's AerSimulator, we were able to observe the noiseless, ideal condition simulation. On the other hand, to simulate the real-world quantum noise, we used Qiskit's `NoiseModel()` with 2% depolarizing error on a two-qubit CX gate.

```
# Create Noise Model (2% depolarizing on CX)
noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.02, 2), ["cx"]
)
```

We ran the simulation sampler with 10,000 shots for $\text{node} = 5$ and 10. However, for $\text{node} = 20$, due to Google Colab's limitations in running bigger depths and sizes of graphs, we had to decrease the number of shots. The number of shots helps in getting a valuable measurement for the histogram of the circuit and depth, meaning the more shots, the better and accurate the results are. Results were saved as bitstring output distributions, and visual outputs were saved as histograms and cost function convergence plots to track optimization performance across iterations. All the simulations were executed using the AerSimulator primitive, with and without noise, and included the fake backend of Qiskit to simulate the real-world NISQ hardware conditions.

```
# Run samplers
sampler = Sampler()
sampler_noisy = Sampler(run_options={"noise_model": noise_model})

job = sampler.run([optimized_circuit], shots=10000)
result = job.result()
quasi_dist = result.quasi_dists[0]

job_noisy = sampler_noisy.run([optimized_circuit_noisy], shots=10000)
result_noisy = job_noisy.result()
quasi_dist_noisy = result_noisy.quasi_dists[0]

[ ] # Convert to bitstrings
def to_bitstring(integer, num_bits):
    return [int(d) for d in np.binary_repr(integer, width=num_bits)]

def process_distribution(dist):
    final_dist = {}
    for k, v in dist.items():
        if isinstance(k, str):
            final_dist[int(k, 2)] = float(v)
        else:
            final_dist[k] = float(v)
    return final_dist

final_dist = process_distribution(quasi_dist)
final_dist_noisy = process_distribution(quasi_dist_noisy)
```

1.5 Classical Optimization

As for the optimization of QAOA, we used parameters using the COBYLA optimizer from `scipy.optimize.minimize`, which is a gradient-free method that is suitable for noisy optimization problems. For $p = 1$, the initial parameters were set to $[\pi/2, \pi/2]$. The optimization then minimized the negative expected value of the cut, which maximized the probability of high-value cuts.

1.6 Classical Benchmarking

```
X = cp.Variable((n, n), symmetric=True)
constraints = [X >= 0]
for i in range(n):
    constraints.append(X[i, i] == 1)

objective = cp.Maximize(0.25 * cp.sum(cp.multiply(A, (1 - X))))
problem = cp.Problem(objective, constraints)
problem.solve()

np.float64(5.18549243862834)
```

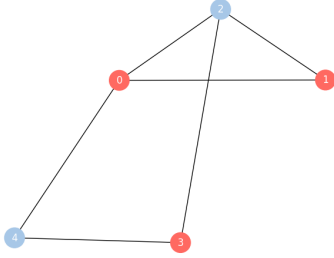
To compare the performance of this algorithm, we also implemented the classical method to solve the Max-Cut problem using the Goemans-Williamson algorithm, utilizing the `cvxpy` library, which solved the semidefinite programming SDP relaxation of the Max-Cut problem and gave about 87.8% approximation ratio. An example of this is implemented using the 5-node graph, where this algorithm gave a cut value of 5, which will help in comparison to the quantum approach using QAOA under different levels of depths and with simulation of noise.

```
[ ] def sdp_rounding_rx(X_opt, graph, num_trials=500):
    best_cut = 0
    best_partition = None
    n = X_opt.shape[0]
    jitter = 1e-5
    try:
        L = np.linalg.cholesky(X_opt + jitter * np.eye(n))
        V = L.T
    except np.linalg.LinAlgError:
        eigvals, eigvecs = np.linalg.eigh(X_opt)
        V = np.diag(np.sqrt(np.maximum(eigvals, 0))) @ eigvecs.T
    for _ in range(num_trials):
        r = np.random.randn(n)
        r /= np.linalg.norm(r)
        partition = np.sign(V @ r)
        partition[partition == 0] = 1
        cut_value = 0
        for i, j in graph.edge_list():
            if partition[i] != partition[j]:
                cut_value += 1
        if cut_value > best_cut:
            best_cut = cut_value
            best_partition = partition
    return best_cut, best_partition

X_opt = X.value
cut_value, partition = sdp_rounding_rx(X_opt, graph)
print(f"SQP Optimal Value (Upper Bound): {problem.value:.4f}")
print(f"Best Rounded Cut Value found: {cut_value}")

SQP Optimal Value (Upper Bound): 5.1855
Best Rounded Cut Value found: 5

Goemans-Williamson Cut (Value: 5)
```



1.7 Supporting Code and Visualization

We visualized the results by plotting the output bitstring probabilities using Qiskit's `plot_histogram` function, and generated energy cost convergence plots to show how the optimizer minimized the cost over time. We determined the most probable Max-Cut solution by identifying the bitstrings with the highest measurement probability, like [1, 1, 0, 1, 0] and validating it against the known optimal cut of the graph. An example is using the initial implementation using a nodes = 5 graph, the bitstring gave a valid max-cut value of 5, which matches the optimal cut and confirms the correctness of the result of noiseless QAOA implementation.

```
[ ] def evaluate_sample(x: Sequence[int], graph: rx.PyGraph) -> float:
    assert len(x) == len(list(graph.nodes()))
    cut_value = 0
    for u, v in graph.edge_list():
        cut_value += x[u] * (1 - x[v]) + x[v] * (1 - x[u])
    return cut_value

cut_val = evaluate_sample(most_likely_bitstring, graph)
print(f"The value of the cut is: {cut_val}")

The value of the cut is: 5
```

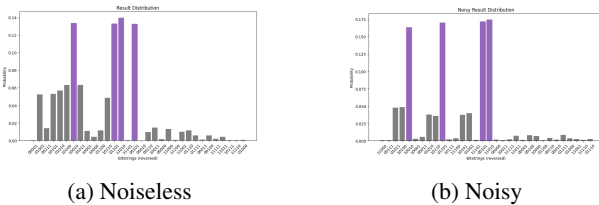


Fig. 4: QAOA output probabilities for depth = 2, nodes = 5 (noiseless vs noisy).

1.8 Reproducibility

All implementations were written in Python 3.10 using the latest version of Qiskit. The Google Colab environment included dependencies such as qiskit, networkx, matplotlib, scipy, and rustworkx. Results were run using seeded simulations to allow for reproducibility. Full source code, data, outputs, and circuit visualizations are available in our shared team repository: <https://github.com/annaseghal/QAOA-implementation>.

1.9 Implementation Strategy and Optimizer Choice

For the implementation of QAOA, we applied QAOA with a scaling-up the depth strategy and increased the number of layers in each iteration. We first tested on a smaller graph, for example, 5 nodes, and these graphs were converted into a cost Hamiltonian derived from the adjacency matrix, and then a scoring function was created for all the possible edges ("cuts") in the graph. The quantum circuit was trying to find the best possible bitstring that connects the most edges between the partitions. We used the Qiskit AerSimulator due to the usage limit of the newer version of the IBM quantum platform, for speed, and built-in noise and noiseless model comparison. Simulation also helps reduce the possibility of hardware decoherence and gate error, and adding simulated noise using `NoiseModel()` helped in getting the test's practical application. We used the fake backend to give similar device behavior to a hardware device. For the classical optimization, we used COBYLA, and researched SPSA optimization, which was not implemented for this project due to limitations newer documentation of AerSimulator noise-simulation. COBYLA is a cautious and gradient-free optimization method that works well for ideal, noiseless simulations. On the other hand, SPSA works well with a noisy environment, which still manages to converge, even when we don't have the exact gradients. Running COBYLA and SPSA would give us an idea of how QAOA behaves in different environments. However, for this project, we only used COBYLA to run the noisy optimization.

```
print("\n Optimizing Noiseless QAOA...")
result = minimize(qaoa_objective, init_params, args=(estimator_noiseless,), method="COBYLA")

print("\n\n Optimizing Noisy QAOA...")
result_noisy = minimize(qaoa_objective, init_params, args=(estimator_noisy,), method="COBYLA")
```

2 ANALYSIS

Through this project, we focused on the QAOA's behavior related to circuit depth, noiseless and simulated noise, and compared it against the classical Goemans-Williamson algorithm.

2.1 Circuit tracing

The QAOA circuit operation starts with the cost problem application and mixer layers, which start from a superposition state. For our circuit we will be using depth = 1, and nodes = 5 for this analysis.

we start by initializing all the qubits to the $|0\rangle$ state: q_0, q_1, q_2, q_3, q_4 . Each is equal to

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Then we apply the Hadamard H gate to each of those qubits to get the superposition:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = |+\rangle$$

This for all qubits, or a bigger circuit with n qubits, gives us a state where all 2^n bitstrings (like $|00000\rangle, |00001\rangle, \dots, |11111\rangle$) have equal amplitude. For our circuit, 5 bits, it would be equal to $2^5 = 32$ possible bitstrings, which has an amplitude of $1/\sqrt{32}$.

After the initial state, we have the Cost Unitary Layer $U_C(\gamma)$. This particular layer changes the angle, also known as the phase, of each of the bitstrings based on how well it solves the Max-Cut problem. The changes are based on the problem Hamiltonian, which is the sum of the Pauli-ZZ for each edge, including terms like $Z_u Z_v$ for each edge in the graph. One example of this is for state $|00111\rangle$, it might look like:

$$0.126230 \times e^{i0.122540} |00111\rangle$$

This means the bitstring now has a complex amplitude with magnitude 0.126230 and a phase of 0.122540 radians.

Finally, we have the Mixer layer $U_M(\beta)$. This layer allows the system to get new combinations by rotating each of the qubits using angle β . These rotations will then help the quantum state to mix amplitudes between different computational basis states and try to move the state closer to the optimal solution. The parameters γ and β are optimized by a classical computer to improve the result. This process is then repeated for the chosen circuit depth, and finally measures the quantum state at the end and collapses to one specific bitstring, which is ideally one that gives the optimal solution for the Max-Cut.

2.2 Effect of Noise and Depth

```

--- Noiseless Simulation Results ---
Optimized Cost Table (Noiseless):
  5 Nodes  10 Nodes  20 Nodes
p=1  -2.888  -1.309  -11.156
p=2  -2.215  -1.441  -11.882
p=3  -2.268  -6.984  -17.842

Max-Cut Value Table (Noiseless):
  5 Nodes  10 Nodes  20 Nodes
p=1      5      8      31
p=2      5      8      31
p=3      5     15      31

--- Noisy Simulation Results (2% CX Error) ---
Optimized Cost Table (Noisy):
  5 Nodes  10 Nodes  20 Nodes
p=1  -2.281  -5.287   -3.8
p=2  -1.328  -5.179   -1.8
p=3  -2.275  -5.168   -3.8

Max-Cut Value Table (Noisy):
  5 Nodes  10 Nodes  20 Nodes
p=1      5     11     12
p=2      5     11     12
p=3      5     11     12

```

In our project, we can see the interplay between the **depth** ($p = 1, 2, 3$) and the **impact of noise** on the performance of the QAOA for the **Max-Cut problem**. In a **noiseless simulation**, increasing the depth of the circuit, such as $p = 1, 2, 3$, meaning that depth = 3, would yield **better results** than depth 1.

For the graph sizes we worked with, like **5, 10, and 20 nodes**, a deeper circuit would get a **more negative optimized cost value**, which means it has **better energy minimization** and usually is matched or improved through the Max-Cut value. For **node = 5** and **node = 20** graphs, QAOA gave the **optimal Max-Cut values** of 5 and 31, across all depths. For **node = 10**, depth $p = 3$ lets QAOA get a **better solution** (cut value of 11) compared to the 8 found at lower depths.

This means in an **ideal, noiseless condition**, increasing depth **improves** the ability of the QAOA algorithm to **find solutions** and reach **near-optimal solutions**.

Nevertheless, when **noise is introduced** (simulated with a **2% CX gate error**), **performance degradation** mainly occurs for **larger graphs** and **deeper circuits**. For the **5-node graph**, QAOA worked well; it found the **best Max-Cut score of 5**, even under **noisy conditions** across all depths, which means that under noise, **smaller or simpler circuits** can still work well.

In addition, for a graph with **10 nodes**, a **noisy simulator** could get the Max-Cut value of 11, similar to the noiseless case, especially for **depth $p = 3$** . This means the **noise does not always cause issues** for the performance of QAOA, and depending on the **structure of the circuit** or **graph behavior**, it can still give an **optimal solution**.

Lastly, for a graph with **20 nodes**, we can see a **drop in performance**, partly due to **Google Colab's limitations** on running many shots. It can be noticed due to the **big drop** in the solution under noise compared to the **noiseless optimal solution**. Even though the **noiseless simulation** gives the **optimal Max-Cut value of 31**, **noisy simulations** aren't able to reach that due to **usage eliminations**, as we can see in $p = 1$ and $p = 3$.

This **degradation** shows how both **increase in size** and **circuit depth**, combined with **errors and decoherence** under noisy conditions, can **hurt the QAOA's search** for the cut. This points to the need for ways to **cut down errors** or **improve hardware** to help QAOA work well on a **larger scale**.

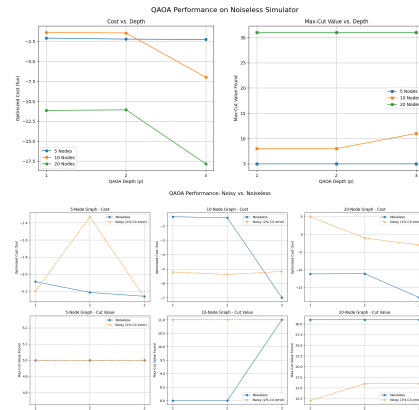
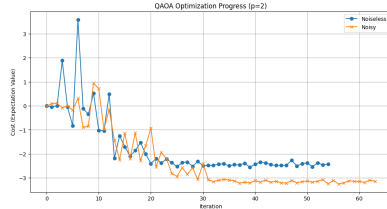


Fig. 5: Cost function gap between noisy and noiseless runs across circuit depths for different graph sizes. As depth and size increase, noise sensitivity increases.

2.3 Optimizer Behavior

The optimizer **COBYLA**, in this case, **consistently met the mark**, as we can see in the **plots of the function**. One example of this is a **5-node graph** with $p = 2$ that converged to a cost of **-2.73828125**. It did work well, but the **noise affected the optimization paths**, making them **less smooth**. At times, this led to **suboptimal answers**, especially as the **problem size increased**.



2.4 Benchmarking vs Goemans-Williamson

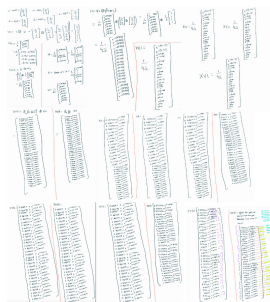
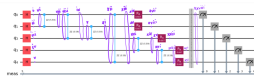
For the **noiseless simulation**, QAQA was able to get the **optimal cut values** for the **5-node graph** and the **theoretical optimal solutions** for **10 and 20-node graphs**, which shows the ability to **reach or exceed the classical approximation method** in **ideal conditions**.

On the other hand, under **noisy conditions**, as the graph grows especially for a **20-node graph** the QAQA **slows down** in the simulator and is **outperformed** by the classical **Glauber-Wang algorithm** from our observation. This points out a limitation of the **current quantum hardware** and the necessity of **robust error mitigation** for achieving **practical quantum advantage**.

Graph Size	(p)	QAQA (Noisy)	GW Score
5 Nodes	2	5	5

2.5 Mathematical tracing

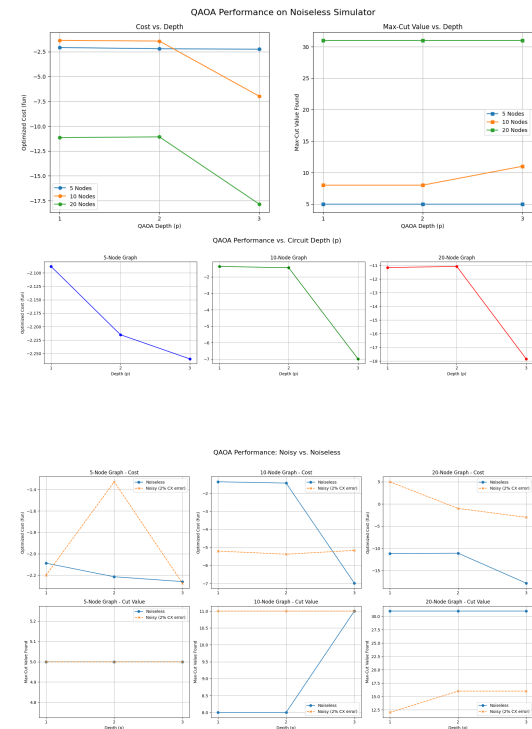
The circuit tracing step-by-step-clearer-version of tracing: <https://tinyurl.com/y73xbdnj>



3 EXTENSION

We extended our base QAQA circuit by **varying the quantum depth parameter (p)** and evaluated its performance under both **noisy and noiseless conditions** using simulated (fake) backends. **This is a meaningful extension to the basic circuit**, as it helps us assess the practical challenges of using QAQA on real-world quantum hardware.

For the first extension, we implemented QAQA into varying depths ($p = 1, 2, 3$) and tested the circuit as the depth increased. One of our observations was that as the depth increases, the better quality solution we get in both ideal, noiseless, and noisy environments. For depth $p = 1$, we observed that the final minimal costs, which are the expected values of the QAQA cost function, remained relatively close between the noisy and noiseless simulations, even as we scaled the graph size up to 20 nodes. **The average difference in minimal cost between the two environments stayed within approximately ~ 0.4 to 0.6** , suggesting that shallow QAQA circuits are **somewhat robust to moderate noise at this scale**.



Our second extension was adding a noise model simulator using the Qiskit AerSimulator backend to simulate quantum noise due to the usage limit of the current hardware. **When we increased the QAQA depth to $p = 3$** , we observed a dramatic increase in sensitivity to noise. The difference between the noisy and noiseless final costs widened significantly: at 5 nodes, the cost gap grew to around 0.6, but at 20 nodes, this difference expanded to approximately 9.0. This reflects the cumulative effect of gate noise, decoherence, and circuit depth—as the number of qubits and entangling operations increase, so does the algorithm's susceptibility to noise, resulting in signifi-

cantly degraded performance in realistic conditions. In addition, we also experimented on different graph sizes, such as 5, 10, and 20-node graphs, to see the effects of graph complexity being increased as depth and noise increased; **we noticed this as we ran a 20-node graph on the fake backend, where it kept getting disconnected and was significantly slower.**

Furthermore, we also **analyzed the impact of the Hadamard gate specifically at the initialization of the quantum state.** Removing a single Hadamard gate reduced performance by limiting the circuit's ability to explore the full solution space. When all Hadamard gates were removed, the algorithm failed, as it could no longer generate a superposition of states. These results emphasize the importance of circuit initialization and how even minor structural changes can affect QAOA's effectiveness.

Our findings help us evaluate the limitations and capabilities of QAOA in a real-world setting as we integrate noise and as depth increases. We can see that while QAOA performs well at low depths or small problem sizes, running deeper circuits on noisy hardware remains a challenge. Effective noise mitigation and careful circuit design are necessary to unlock the full potential of QAOA on larger-scale optimization problems.

4 CONCLUSION

To sum it all up, we were able to implement and test the Quantum Approximate Optimization Algorithm for the NP-hard challenge Maxcut on different graph sizes with nodes of 5, 10, and 20 using the Qiskit AerSimulator. The results for the ideal, noiseless condition become better as circuit depth increases.

On the other hand, with the addition of noise simulating the current state of hardware in the real world, the performance of the QAOA algorithm further degrades as the graph G increases. Compared to the known Goemans-Williamson algorithm, QAOA can be as good or better than classical methods in a noiseless and ideal environment. However, in real-world conditions, where QAOA is under noisy conditions, the performance is less effective, allowing Goemans-Williamson to remain a strong algorithm and outperform it.

For smaller graphs, QAOA works well even with noise, giving good results, and gets close in performance to the traditional approach in noiseless benchmarks. One takeaway is how noise plays a crucial role in the current state of hardware and affects algorithms like QAOA. Further research into error mitigation and dealing with quantum noise would yield significant results. Achieving a success rate that rivals the GW algorithm's would be particularly fruitful with additional error mitigation. In addition, it shows that QAOA has big promise with soon-to-come advancements, but also points out the need for strong ways to deal with noise. Such steps are needed to match the performance of classical algorithms like Goemans-Williamson.

5 REFERENCES

- [1] IBM Quantum, "Quantum Approximate Optimization Algorithm (QAOA) Tutorial," doi:<https://tinyurl.com/3ujswnuy>
- [2] Qiskit, "QAOA — Quantum Approximate Optimization Algorithm — Qiskit," YouTube, doi:<https://www.youtube.com/watch?v=P3s7TIMIvZ0>
- [3] IBM Quantum, "Qiskit Primitives API," doi:<https://quantum.cloud.ibm.com/docs/en/api/qiskit/primitives>
- [4] IBM Quantum, "Qiskit AerSimulator API," doi:<https://quantum.cloud.ibm.com/docs/en/api/qiskit/0.30/qiskit.providers.aer.AerSimulator>
- [5] A. Verma, "Understanding Max-Cut and QAOA," doi:<https://www.avneeshverma.com/Maxcut/maxcut.html>
- [6] G. Gaurav, "QAOA Explained with Max-Cut Example," Medium, doi:<https://medium.com/@gerrard.gaurav/qaoa-explained-with-max-example-632a3f101f12>
- [7] Qiskit, "How Does QAOA Work?," YouTube, doi:<https://www.youtube.com/watch?v=cMZcA2SQnYQ>
- [8] Classiq, "QAOA Max-Cut Implementation," GitHub, doi:https://github.com/Classiq/classiq-library/blob/main/algorithms/qaoa/maxcut/qaoa_max_cut.ipynb
- [9] PennyLane, "QAOA Introduction Tutorial," doi:https://pennylane.ai/qml/demos/tutorial.qaoa_intro
- [10] T. Lawrence, "Quantum Approximate Optimization Algorithm Explained," Medium, doi:<https://tinyurl.com/56w5jt8z>
- [11] IBM Quantum, "Utility-Scale QAOA," doi:<https://quantum.cloud.ibm.com/learning/en/courses/quantum-computing-in-practice/utility-scale-qaoa>
- [12] M. X. Goemans and D. P. Williamson, "Improved Approximation Algorithms for Maximum Cut and Satisfiability Problems Using Semidefinite Programming," *J. ACM*, vol. 42, no. 6, pp. 1115–1145 (1995). doi:<https://math.mit.edu/~goemans/PAPERS/maxcut-jacm.pdf>
- [13] NVIDIA, "QAOA for Max-Cut - Recursive Divide and Conquer," GitHub, doi:https://github.com/NVIDIA/cuda-q-academic/blob/main/qaoa-for-max-cut/03_Recursive-divide-and-conquer.ipynb
- [14] Dr. Kevin Kissi, "Quantum Approximate Optimization Algorithm - Concept and Implementation," YouTube, doi:<https://www.youtube.com/watch?v=aFVnWq3RHYU>
- [15] IBM Quantum, "Building Noise Models," doi:<https://quantum.cloud.ibm.com/docs/en/guides/build-noise-models>
- [16] Qiskit Aer, "qiskit.aer.noise.NoiseModel Documentation," doi:<https://qiskit.github.io/qiskit-aer/stubs/qiskit.aer.noise.NoiseModel.html>
- [17] Entropica Labs, "What is the QAOA?," OpenQAOA, doi:<https://openqaoa.entropicalabs.com/what-is-the-qaoa>

6 TEAM CONTRIBUTION

- **Anna Sehgal** - Led the circuit implementation and simulation using Aersimulator. Developed and managed the collaborative notebook implementation and exten-

sion implementation in Collab Notebook, performed key research on QAOA structure and optimization, and contributed to core project planning and problem formulation. Played a central role in circuit tracing/analysis, testing, and visualizations. Impclassical classical benchmarking.

- **Alexander L. Liu** - Focused on QAOA circuit extensions and experimental variations. Contributed detailed notes and theoretical analysis regarding depth variation, noise models, and circuit modifications. Helped frame the extension section and deepen the experimental scope.
- **Jasjit Dhillon** - Led the introductory framing of the project, including the overview of QAOA, Max-Cut, and classical benchmarking. Drafted early slides, supported overall documentation, and contributed to initial project planning discussions and presentation strategy.
- **Matthew J. Hernandez** - Focused on methods and implementation structure. Researched optimization techniques, noise models, and backends. Documented the methodology and experimental pipeline, ensuring clarity in how simulations were set up and executed.